

Семантический анализатор

Виды семантики

- Операционная
- Денотационная
- Аксиоматическая

Операционная семантика

- Операционная семантика описывает значение программы через последовательность вычислительных шагов, имитируя ее выполнение на абстрактной или виртуальной машине.
- Фокусируется на том, как состояние программы (память, регистры) изменяется от шага к шагу.

Подвиды операционная семантика

- Трансляционная
 - Определяет значение программы путем ее перевода в другой язык, чья семантика известна, сохраняя исходное значение.
 - Фокус на перевод в целевой язык.
- Трансформационная
 - Описывает значение через последовательные преобразования или переписывания программы в более простую форму.
 - Фокус на преобразовании программы.

Денотационная семантика

- Присваивает математические объекты (денотации) такой как функция или множество, определяя значение как композицию этих объектов.
- Она абстрагирует от шагов выполнения, фокусируясь на входе-выходе.
- Подходит для абстрактного анализа, хотя может быть сложной для императивных языков.

Аксиоматическая семантика

- Использует логические утверждения (аксиомы и правила вывода) для описания свойств программ, таких как пред- и постусловия.
- Она не описывает полное выполнение, а фокусируется на доказательствах корректности.
- Идеальна для верификации.

Атрибутивная грамматика

Атрибутивная грамматика

- Расширение КС грамматики
- Придумана Дональдом Кнутом в 1968 году для формализации семантики контекстно-свободных языков

Структура атрибутивная грамматика

- правила грамматики
- семантические действия
- для каждого символа грамматики могут задаваться атрибуты
- два способа передачи значений атрибутов:
- наследование
- синтезирование

Пример

```
Expr → Term Elist { Expr.val = Term.val + Elist.val }
```

```
Elist → + Term Elist1 { Elist.val = Term.val + Elist1.val }  
      | ε { Elist.val = 0 }
```

```
Term → Factor Tlist { Term.val = Factor.val * Tlist.val }
```

```
Tlist → * Factor Tlist1 { Tlist.val = Factor.val * Tlist1.val }  
      | ε { Tlist.val = 1 }
```

```
Factor → num { Factor.val = num.val }
```